FABRIC / PRACTITIONER SERIES

SESSION 03 / TEACHING HANDBOOK

Low-code Fabric Data Engineering

From source extracts to an explainable, repeatable, governed analytical workload.

October 28, 2026
SQL and Power Query practitioners

06	180	SQL-first
DEVELOPED LESSONS	MINUTES OF LEARNING	PRACTICAL DELIVERY

CONCEPTS / WORKED EXAMPLES / PRACTICE / OPERATIONS

YOUR LEARNING PATH

From decision to delivery

One reconciliation workload connects the lessons. Follow the decisions, engineering patterns, and operating controls from source to consumer.

01 **Architecture decision framework** 03
25 minutes

02 **Repeatable ingestion** 09
35 minutes

03 **Curated SQL engineering** 15
50 minutes

04 **Consumption architecture** 25
35 minutes

05 **Operations and GCC High findings** 31
25 minutes

06 **Exit review** 37
10 minutes

How to use this handbook

Read the explanations, inspect the worked examples, and complete each lesson's output. Contents and lesson links are clickable; PDF bookmarks provide section-level navigation.

Environment and execution boundary

Examples use synthetic data and an isolated training Warehouse. Validate live paths in the target GCC High tenant. General documentation is not proof of tenant availability. SQL examples require preflight before execution.

LESSON 01 / 25 MINUTES

Architecture decision framework

> **WHAT YOU WILL PRODUCE**
>
> A completed architecture decision record.

What this lesson teaches

A good architecture is not the one that contains the most Fabric components. It is the one that meets the business requirement with a clear transformation path, appropriate controls, and an operating model the team can sustain.

In this lesson, we will revisit the reconciliation workload from Session 2, compare Warehouse and Lakehouse, and make a deliberate starting-point decision. The objective is not to memorize a product comparison. It is to learn how to turn a business requirement into a defensible technical choice.

Use approximately five minutes to revisit the workload, eight minutes to compare the platforms, seven minutes to select an architecture, and five minutes to record the decision.

1. Start with the business question

Our workload begins with two systems that describe overlapping business records. Each system produces a file, and those files are stored in SharePoint. Their column names and representations differ. One system may call a field `RecordId`, while the other calls it `Reference`. An identifier might contain surrounding spaces or inconsistent capitalization. An amount might be a valid number in one file and an invalid text value in the other.

The business question is straightforward:

> Do the two extracts describe the same records, amounts, and statuses? If not, which differences require investigation?

This is more precise than saying, "We need to move some files into Fabric." Moving files is an implementation activity. Reconciliation is the business outcome.

The solution must preserve source evidence, make comparable values consistent, identify differences, and present exceptions to an approved audience. It must also repeat successfully when the next extract arrives. A one-time result that depends on someone manually repairing every refresh does not yet satisfy the operating requirement.

Before selecting a platform, identify the **grain** of the output. Grain describes what one row represents. For this workload, one published reconciliation row represents one normalized business record ID in one comparison of two complete source snapshots. If a source contains duplicate IDs, the solution must identify that ambiguity rather than accidentally multiplying rows during a join.

The intended consumer is a Power BI audience that needs a trusted summary and enough detail to investigate exceptions. SQL practitioners also need a way to inspect the underlying curated data. Those consumption needs influence how we publish the output, but they do not require every possible Fabric engineering tool.

2. Understand what Warehouse and Lakehouse share

Warehouse and Lakehouse are both Fabric analytical data stores built on OneLake and open Delta storage. Choosing Warehouse does not mean choosing something outside OneLake. Choosing Lakehouse does not mean giving up relational queries.

The important differences concern the development experience, supported write operations, data organization, and the team's transformation approach.

A **Warehouse** is a strong starting point when the work is primarily relational and the team intends to create and modify analytical tables using T-SQL. It supports Warehouse tables, views, stored procedures, and supported data manipulation operations. It also supports transactions that can group related SQL changes into a single commit or rollback.

A **Lakehouse** is a strong starting point when the workload needs file-oriented engineering, structured and unstructured data together, or Spark-based transformation. Its SQL analytics endpoint lets SQL users query exposed Delta tables and define supported SQL objects. However, that endpoint does not turn Lakehouse tables into tables that can be inserted, updated, or deleted through T-SQL.

This distinction is especially important for SQL developers. Seeing a SQL query editor does not establish that the underlying item supports the same write path as a Warehouse.

Question	Warehouse consideration	Lakehouse consideration
How will we develop transformations?	Primarily T-SQL	Often Spark, notebooks, or other supported Lakehouse writers
What data must we handle?	Relational analytical tables	Files, Delta tables, and potentially unstructured data
Do we need SQL-managed table writes?	A natural fit	Not through the Lakehouse SQL analytics endpoint
Do related SQL writes need a transaction?	Supports multi-table SQL transactions	Do not assume the same cross-table SQL transaction model
What can this team operate today?	Builds on SQL engineering skills	May require additional engineering skills and support practices

Neither product is a complete replacement for every SQL Server capability. Validate required T-SQL features rather than assuming that a familiar syntax or tool guarantees compatibility.

3. Choose the simplest justified architecture

For the current workload, the strongest starting point is a Warehouse-centered design:

```text
SharePoint extracts
        |
        v
Dataflow Gen2
        |
        v
Warehouse: source-aligned landing tables
        |
        v
T-SQL normalization and reconciliation
        |
        v
Curated reconciliation table
        |
        v
Shared semantic model and Power BI
```

This recommendation follows the requirements. The files are tabular. The core transformation is a relational comparison. The audience already knows SQL and Power Query. Nothing in the current requirement demands Spark or a separate notebook tier.

Adding a Lakehouse solely because Fabric has a Lakehouse experience would introduce another component without solving a demonstrated problem. Conversely, if the workload later includes large collections of documents, custom Python processing, or an established Spark engineering pipeline, Lakehouse may become appropriate. Architecture decisions should be revisited when their assumptions change.

The same reasoning applies to notebooks. A notebook is an engineering tool, not a maturity badge. Use one when its capabilities solve a requirement that the chosen SQL and Power Query approach does not address well. Do not introduce Python merely to recreate a straightforward SQL join for an audience that will support the solution in SQL.

Where bronze, silver, and gold fit

Bronze, silver, and gold describe responsibilities in the data flow.

Bronze preserves source-aligned evidence. We should be able to determine what each source supplied, including duplicates and invalid values.

Silver makes values consistent and evaluates quality. Here we align column names, normalize identifiers, convert amounts deliberately, and identify records that cannot be compared safely.

Gold publishes the business-facing result. In this workload, that is an explainable reconciliation classification for each business key, along with the values needed to investigate an exception.

These responsibilities do not automatically require three workspaces, three capacities, or three physical copies of every table. In the training implementation, separate schemas inside one Warehouse make the responsibilities visible. Silver can use views where materialization is unnecessary. Production boundaries should follow security, lifecycle, performance, and ownership requirements.

4. Record the decision and the alternatives

An architecture decision record is a short explanation of why an option was selected under specific conditions. It is useful precisely because the conditions may change.

A useful decision is more specific than "Use Warehouse." It names the workload, the required operations, the assumptions, and the alternatives considered.

Worked decision record

Field	Example decision
Business outcome	Compare two source extracts and expose migration/reconciliation exceptions
Data and grain	Structured snapshots; one result per normalized record ID per comparison
Selected starting point	Dataflow Gen2 ingestion into a Fabric Warehouse
Transformation approach	Power Query for ingestion shaping; T-SQL for normalization, quality rules, and reconciliation
Consumption approach	Curated table exposed through a shared semantic model; approved SQL access where needed
Reason	Meets the workload using the team's existing SQL and Power Query skills
Alternative considered	Lakehouse with notebook-based transformation
Why not selected now	No current file/Spark requirement justifies a second engineering path
Assumptions	Approved source access, functioning destination connections, and verified tenant capabilities
Revisit when	Data types, transformation requirements, scale characteristics, security boundaries, or team ownership change

Notice that an unverified capability is an assumption to test, not a fact to hide. The engagement's prior experience is useful evidence, but it does not establish that every feature or connection works today.

5. Apply the lesson

Write a decision record for the reconciliation workload using the fields above. Then change one assumption: the source now includes documents requiring custom Python extraction before comparison. Explain whether that change affects ingestion, transformation, storage, or all three.

A strong answer does not automatically replace the entire solution. It identifies where the new requirement belongs and preserves the parts that still meet their purpose.

Check your understanding

Why is Warehouse a reasonable starting point here? The workload is relational, requires supported SQL writes, and is operated by a SQL/Power Query-oriented team.

Does Warehouse bypass OneLake? No. Warehouse also uses OneLake and Delta storage.

Can we write Lakehouse table data through its SQL analytics endpoint? No. Query access and supported SQL object definitions are not the same as T-SQL table-data writes.

Does a medallion pattern require three workspaces? No. The layers identify responsibilities; physical separation requires its own justification.

Lesson conclusion

We have selected a starting architecture based on a business question, output grain, required operations, and team capability. The next lesson turns that architecture into a repeatable ingestion contract.

Next: Lesson 2: Repeatable ingestion

Product references: Warehouse/Lakehouse decision guide and T-SQL surface area. General product documentation does not confirm availability in the target GCC High tenant.

LESSON 02 / 35 MINUTES

Repeatable ingestion

> **WHAT YOU WILL PRODUCE**
>
> A repeatable ingestion pattern with its scheduling method documented.

What this lesson teaches

Ingestion is the controlled movement of source data into a location where it can be processed reliably. A successful connection is only the beginning. We must also know which files were selected, how their values were interpreted, where their rows were written, and what another execution will do.

This lesson uses Dataflow Gen2 because it builds on familiar Power Query skills. We will preserve source evidence, select explicit destinations, distinguish full replacement from incremental processing, and separate scheduling from orchestration.

Allocate approximately five minutes to the source contract, fifteen minutes to the dataflow walkthrough, seven minutes to load strategies, and eight minutes to scheduling and the participant output.

1. Establish a source contract

A SharePoint folder is a storage location, not a complete definition of a dataset. It may contain current files, previous extracts, temporary files, or unrelated documents. If a query combines everything it can see, its results can change when someone adds a file that was never intended to participate.

For this lesson, define two exact inputs in an approved training folder: `system_a.csv` and `system_b.csv`. They are synthetic, complete snapshots of the same comparison scope. They are not employee data and do not contain personal information.

Use the following content when preparing the files.

System A

```csv
RecordId,Amount,Status
A001,100.00,Active
A002,200.00,Active
A003,300.00,Active
A005,500.00,Active
A005,500.00,Active
A006,not provided,Active
```

System B

```csv
Reference,Total,State
 a001 ,100.00,ACTIVE
A002,250.00,ACTIVE
A004,400.00,ACTIVE
A005,500.00,ACTIVE
A006,600.00,ACTIVE
```

System A contains six rows. System B contains five. The duplicate A005 and invalid A006 amount are intentional. They are not errors to remove before the exercise; they are evidence the reconciliation must explain.

The surrounding spaces and lowercase characters in System B's first identifier are also intentional. They let us distinguish a harmless representation difference from a business difference.

A source contract should state the file location and selection rules, expected columns, value formats, snapshot scope, source owner, and the condition that means the extract is ready. If two files are intended to represent the same reporting period, their presence alone does not prove they represent the same period. The source owner must provide that assurance through a controlled delivery convention or explicit batch metadata.

2. Build or inspect the Dataflow Gen2 ingestion

Dataflow Gen2 uses Power Query to connect, select, shape, and load data. This is useful when practitioners want a visible sequence of transformation steps rather than a custom program.

For this workload, keep ingestion focused on a small set of responsibilities: select the intended files, read their tabular structure, preserve their source values, and write them to explicit landing tables.

Connect with the intended execution identity

Use an approved connection to the SharePoint site. Confirm that the connection can read the intended location and that the account or identity used for scheduled execution has the necessary access.

The presenter's ability to open a file in a browser is not proof that a scheduled dataflow can read it. Interactive access and service execution may use different connections, identities, policies, or gateway paths.

Do not add a gateway simply because a connection fails. Establish whether the actual source and network configuration require one. If a gateway is part of the approved path, validate its availability and connection visibility before class.

Filter deliberately

Inspect the file list before combining or opening content. Filter by the intended folder path and file name, and exclude irrelevant or temporary files. Select the expected sheet or table if the real workload uses Excel.

The goal is deterministic input selection: adding an unrelated file to SharePoint must not silently change the dataset.

Preserve text in the landing layer

Power Query often adds type-conversion steps automatically. Inspect them rather than accepting them without review.

For this lesson, all three source columns should land as text. The value `not provided` must reach the Warehouse so SQL can classify it as an invalid amount. If ingestion converts the column to a number too early, the refresh might fail, or a later error-removal step might discard the record.

Removing the row would make the dataset look cleaner while making the reconciliation less truthful. Bronze is the place to preserve evidence, not conceal quality problems.

Retaining raw text does not mean that everything stays text forever. It means that type conversion is performed deliberately in the next layer, where both the converted value and the quality outcome can be inspected.

Select explicit destinations and mappings

Use the prepared Warehouse landing tables from Lesson 3, created before the walkthrough:

Source	Destination	Mapping
System A	`bronze.system_a`	`RecordId`, `Amount`, `Status` to matching text columns
System B	`bronze.system_b`	`Reference`, `Total`, `State` to matching text columns

A preview in the Power Query editor is not the same as a completed load. Inspect the destination configuration for each output query. Confirm the Warehouse, schema, table, column mapping, and update method.

For these complete snapshots, use the validated **replace** behavior for the existing landing tables. Do not assume a managed-settings option described for another destination applies to Warehouse. Inspect the actual settings available in the target environment.

Publish or save the dataflow using the available tenant experience, run it, and inspect the actual destination data after successful completion. Expected counts are six rows in System A and five in System B.

3. Understand full replacement, append, and incremental loading

These terms describe different behaviors and should not be used interchangeably.

Full replacement treats the incoming dataset as the current complete state. Each successful load replaces the previous snapshot. Loading the unchanged files again should leave six and five landing rows.

Append adds incoming rows to the rows already present. Appending the same complete snapshots twice would produce twelve System A rows and ten System B rows. That is not incremental correctness; it is duplicated input.

Incremental processing identifies and handles changes relative to a previously processed state. It needs more than an append switch. It requires a reliable change indicator or change feed, a key, update/delete handling, a durable checkpoint, and a policy for retries and late arrivals.

Consider an amount changing from 200.00 to 250.00. An append-only load now has both values unless another rule determines which is current. If a record is deleted from the source, a change feed containing only inserts will never tell the destination to remove it. If two changes share a timestamp and a watermark advances too early, a record can be missed.

For small complete reconciliation extracts, full replacement is a reasonable teaching choice because it makes the current-state contract clear. It is not a recommendation to reload every enterprise source indefinitely.

Why two completed loads matter

Replacing each destination independently does not make the pair atomic. System A can finish while System B fails. Running reconciliation at that moment could compare today's System A against yesterday's System B.

The SQL publication step must therefore wait until both sources have completed for the intended snapshot pair. In this lab, that gate can be an explicit supervised check. In production, use a verified control mechanism and durable batch evidence. Do not infer a matched pair merely because both tables contain rows.

4. Separate scheduling from orchestration

A schedule answers, "When should this activity start?" Orchestration answers, "What depends on it, and what happens when it succeeds or fails?"

A direct Dataflow Gen2 schedule may be enough to start ingestion. It does not, by itself, prove that downstream SQL ran, that the semantic model observed the new publication, or that a report displays the intended snapshot.

A pipeline can coordinate a dataflow, a SQL activity, and downstream steps using supported activities and success dependencies. Its value is the explicit dependency chain, not merely providing another clock.

```text
Source pair ready
    -> ingestion succeeds for both destinations
    -> source-pair checks pass
    -> reconciliation succeeds
    -> model freshness step succeeds
    -> publication is verified
```

Scheduling SQL fifteen minutes after ingestion is not equivalent to waiting for ingestion success. Runtime can vary, credentials can fail, and a source can be unavailable.

The lesson plan records a previous pipeline-to-Dataflow authentication or permissions issue and an interim direct-scheduling approach. Retest the actual path before delivery. If the pipeline remains blocked, document ingestion as scheduled and the SQL/model steps as supervised. Do not describe that partial automation as a completed end-to-end orchestration.

Where copy jobs fit

A copy job is worth considering when the primary requirement is supported data movement with limited transformation. Dataflow Gen2 is appropriate here because file selection and Power Query shaping are part of the requirement. A pipeline coordinates activities; a copy job moves supported data; a dataflow shapes and loads data.

The exact source, destination, load mode, and Government availability must be checked before selecting a copy job. Do not replace the working SharePoint/Power Query design based on a product label alone.

5. Document the ingestion pattern

Complete the following contract for the actual training environment.

Contract item	Required answer
Source scope	Approved site, folder, exact files, and comparison period
Readiness	How the source owner identifies a complete, matching snapshot pair
Connection	Approved connection owner and execution identity; no credentials in the document
Query behavior	File filters, expected columns, and text-preserving type rules
Destinations	Warehouse, schemas, tables, and mappings
Load strategy	Replace for the complete lab snapshots
Rerun expectation	Unchanged inputs leave six and five rows
Trigger	Verified direct schedule, pipeline trigger, or explicit manual execution
Downstream gate	Both destinations completed for the intended pair
Failure behavior	Stop downstream publication and retain the last known-good published result

Practice and expected explanation

Run the unchanged load twice if the environment and available time permit. Inspect counts after each run. Explain why the duplicate A005 must still exist in Bronze but why a second refresh must not add another copy of the entire file.

If System B fails, the correct response is not to proceed because System A succeeded. Record the failure, stop downstream publication, and restore a complete matching source pair before continuing.

Lesson conclusion

Repeatable ingestion is defined by a contract, not by the presence of a refresh button. We now know what data is selected, how it is preserved, what a rerun means, and what must happen before curation begins.

Previous: Lesson 1
Next: Lesson 3: Curated SQL engineering

Product reference: Dataflow Gen2 destinations and managed settings. Confirm the applicable destination behavior and scheduling/orchestration paths in the target GCC High tenant.

LESSON 03 / 50 MINUTES

Curated SQL engineering

> **WHAT YOU WILL PRODUCE**
>
> Curated Warehouse objects and explainable reconciliation results.

What this lesson teaches

SQL curation turns source data into a business result that can be inspected, repeated, and trusted. The important work is not writing a join as quickly as possible. It is deciding what values mean, which records are eligible for comparison, and how an exception remains visible.

We will use the two synthetic extracts from Lesson 2. The examples deliberately retain a duplicate key and an invalid amount. A correct solution explains these conditions instead of deleting them to make the result appear successful.

Allocate five minutes to landing data, ten to normalization, ten to quality and comparison logic, fifteen to the procedure walkthrough, and ten to execution and interpretation. Prepare the objects before class; do not spend the lesson typing every statement. The SQL blocks below are reference material for the walkthrough.

Execution boundary: These examples target an isolated Fabric **Warehouse**, not the Lakehouse SQL analytics endpoint. Run each object-definition block separately. Preflight the SQL in the target tenant. The examples replace only the named training result table, but must not be redirected to production objects.

1. Land source-aligned tables

The landing layer should preserve what the sources supplied. If System A calls a column `RecordId`, retaining that name makes it easier to trace the data back to the file. We will align the names in Silver.

Create the three schemas once, executing each statement separately. If a schema already exists, do not recreate it.

SQL

```sql
CREATE SCHEMA bronze;
```

```sql
CREATE SCHEMA silver;
```

```sql
CREATE SCHEMA gold;
```

Create the landing tables before configuring the Dataflow Gen2 destinations in Lesson 2. These are setup statements, not refresh statements; do not run them again against existing objects.

```sql
CREATE TABLE bronze.system_a
(
    RecordId varchar(100),
    Amount varchar(100),
    Status varchar(100)
);

CREATE TABLE bronze.system_b
(
    Reference varchar(100),
    Total varchar(100),
    State varchar(100)
);
```

The lengths and text types are suitable for the controlled synthetic inputs, not a universal source schema. Real data requires explicit length, encoding, precision, and schema-drift decisions.

Load the files through the dataflow, then inspect the evidence:

```sql
SELECT 'A' AS source_system, COUNT(*) AS row_count
FROM bronze.system_a
UNION ALL
SELECT 'B', COUNT(*)
FROM bronze.system_b;

SELECT RecordId, Amount, Status
FROM bronze.system_a
ORDER BY RecordId;
```

Expect six System A rows and five System B rows. A005 must appear twice in System A, and A006 must retain `not provided` as its amount. If these conditions are missing, investigate ingestion before continuing.

2. Normalize mismatched structures

Normalization creates a common vocabulary without pretending the sources were originally identical.

Both sources will expose `record_id`, `amount`, and `status`. We will trim identifier whitespace, uppercase identifiers and statuses, and convert amounts to `decimal(18,2)`. These are business rules for this controlled example. In another domain, identifier case may be significant and must not be changed.

Using a decimal makes monetary comparison explicit. Floating-point approximation is unnecessary for the two-decimal amounts in this lesson. For real sources, define currency, locale, scale, rounding, and tolerance rules before conversion. A conversion to two decimal places is not itself validation that a source had exactly two decimal places.

```sql
CREATE VIEW silver.normalized_records
AS
SELECT
    CAST('A' AS varchar(1)) AS source_system,
    RecordId AS raw_record_id,
    Amount AS raw_amount,
    Status AS raw_status,
    NULLIF(UPPER(TRIM(RecordId)), '') AS record_id,
    TRY_CONVERT(decimal(18,2), NULLIF(TRIM(Amount), '')) AS amount,
    NULLIF(UPPER(TRIM(Status)), '') AS status
FROM bronze.system_a
UNION ALL
SELECT
    CAST('B' AS varchar(1)),
    Reference,
    Total,
    State,
    NULLIF(UPPER(TRIM(Reference)), ''),
    TRY_CONVERT(decimal(18,2), NULLIF(TRIM(Total), '')),
    NULLIF(UPPER(TRIM(State)), '')
FROM bronze.system_b;
```

`TRY_CONVERT` returns a null when a nonblank value cannot be converted. We also explicitly turn blank text into null before conversion. A null amount means that a usable number is unavailable; it does not mean zero.

Raw values remain available beside their normalized counterparts. This is what lets an investigator see that A006 came from `not provided` rather than guessing why a numeric value is missing.

A view stores a query definition, not a separately loaded snapshot of its result. Views are useful here because the small normalization can be reused without another physical table. They will reflect changes in Bronze, which is another reason to control ingestion while reconciliation runs.

```sql
SELECT
    source_system, raw_record_id, record_id,
    raw_amount, amount, raw_status, status
FROM silver.normalized_records
ORDER BY record_id, source_system;
```

The System B value `a001` should now be `A001`. Both source status values should normalize to `ACTIVE`.

3. Detect quality problems before joining

Suppose a key occurs twice in System A and once in System B. A direct join produces two joined rows for that key. If both sides contain duplicates, the multiplication can be larger. Counting those joined rows as business records would change the output grain.

We must therefore evaluate uniqueness before comparison. For this lesson, a duplicate is an exception, even if the duplicate rows have identical amounts. The source owner, not the join implementation, decides whether a duplicate can be ignored.

Create a summary with one row per source and normalized key:

```sql
CREATE VIEW silver.record_summary
AS
SELECT
    source_system,
    record_id,
    COUNT(*) AS source_row_count,
    SUM(CASE WHEN amount IS NULL THEN 1 ELSE 0 END)
        AS invalid_amount_count,
    SUM(CASE WHEN status IS NULL THEN 1 ELSE 0 END)
        AS invalid_status_count,
    CASE WHEN COUNT(*) = 1 THEN MAX(amount) END AS amount,
    CASE WHEN COUNT(*) = 1 THEN MAX(status) END AS status
FROM silver.normalized_records
WHERE record_id IS NOT NULL
GROUP BY source_system, record_id;
```

The use of `MAX` does not choose a winning duplicate. It exposes a value only when exactly one source row exists. For duplicate keys, those projected values remain null and the row count records the ambiguity.

Blank keys are excluded from this summary because an absent identifier cannot serve as a comparable business key. They remain visible in `silver.normalized_records`. Our teaching procedure will block publication when they exist rather than silently losing them.

```sql
SELECT *
FROM silver.record_summary
WHERE source_row_count > 1
   OR invalid_amount_count > 0
   OR invalid_status_count > 0;

SELECT *
FROM silver.normalized_records
WHERE record_id IS NULL;
```

Expect a duplicate summary for System A A005 and an invalid-amount summary for System A A006. The blank-key query should return no rows for the supplied files.

4. Define reconciliation and exception rules

The business result must distinguish at least three ideas: a comparable match, a comparable difference, and a record that cannot safely be compared.

For this lesson, each nonblank key receives one classification in the following order:

1. `DUPLICATE_KEY`: either source has more than one row for the key.
2. `INVALID_AMOUNT`: an existing source row has no usable amount.
3. `INVALID_STATUS`: an existing source row has no usable status.
4. `ONLY_IN_A` or `ONLY_IN_B`: the other source has no record.
5. `AMOUNT_AND_STATUS_MISMATCH`, `AMOUNT_MISMATCH`, or `STATUS_MISMATCH`: both records are usable but differ.
6. `MATCH`: both usable records agree.

Order matters. If a duplicated record also contains an invalid amount, the published primary classification is `DUPLICATE_KEY`. The Silver evidence still exposes the other problem. A single classification is a reporting choice, not a claim that only one issue can exist.

A full outer join retains keys that occur in only one source. An inner join would omit those exceptions entirely and therefore answer a different business question.

5. Publish through a stored procedure

A stored procedure is a named unit of SQL work that can be executed repeatedly. Creating it defines the work; `EXEC` performs the work.

This is useful because the transformation sequence no longer depends on a person remembering which query to run next. It can later be invoked by a validated orchestration activity. A procedure does not create a schedule, repair credentials, or guarantee that the upstream sources are complete.

First create the physical Gold table. We use a table rather than only a view so there is an explicit publication step and a stable object for downstream consumption.

```sql
CREATE TABLE gold.reconciliation
(
    batch_id varchar(64) NOT NULL,
    published_at_utc datetime2(6) NOT NULL,
    record_id varchar(100) NOT NULL,
    a_row_count int NOT NULL,
    b_row_count int NOT NULL,
    a_amount decimal(18,2),
    b_amount decimal(18,2),
    a_status varchar(100),
    b_status varchar(100),
    classification varchar(40) NOT NULL
);
```

The procedure below publishes the current snapshot result. It is not a historical store: each successful execution replaces the previous Gold rows.

```sql
CREATE PROCEDURE gold.usp_refresh_reconciliation
    @BatchId varchar(64)
AS
BEGIN
    SET NOCOUNT ON;

    IF @BatchId IS NULL OR TRIM(@BatchId) = ''
        THROW 50001, 'A comparison batch identifier is required.', 1;

    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS (SELECT 1 FROM bronze.system_a)
           OR NOT EXISTS (SELECT 1 FROM bronze.system_b)
            THROW 50002, 'Both teaching snapshots must contain rows.', 1;

        IF EXISTS
        (
            SELECT 1
            FROM silver.normalized_records
            WHERE record_id IS NULL
        )
            THROW 50003, 'Blank record keys require investigation.', 1;

        DELETE FROM gold.reconciliation;
```

```
;WITH a AS
(
    SELECT *
    FROM silver.record_summary
    WHERE source_system = 'A'
),
b AS
(
    SELECT *
    FROM silver.record_summary
    WHERE source_system = 'B'
)
INSERT INTO gold.reconciliation
(
    batch_id, published_at_utc, record_id,
    a_row_count, b_row_count, a_amount, b_amount,
    a_status, b_status, classification
)
SELECT
    @BatchId,
    SYSUTCDATETIME(),
    COALESCE(a.record_id, b.record_id),
    COALESCE(a.source_row_count, 0),
    COALESCE(b.source_row_count, 0),
    a.amount,
    b.amount,
    a.status,
    b.status,
    CASE
        WHEN a.source_row_count > 1 OR b.source_row_count > 1
            THEN 'DUPLICATE_KEY'
        WHEN COALESCE(a.invalid_amount_count, 0) > 0
          OR COALESCE(b.invalid_amount_count, 0) > 0
            THEN 'INVALID_AMOUNT'
        WHEN COALESCE(a.invalid_status_count, 0) > 0
          OR COALESCE(b.invalid_status_count, 0) > 0
            THEN 'INVALID_STATUS'
        WHEN b.record_id IS NULL THEN 'ONLY_IN_A'
        WHEN a.record_id IS NULL THEN 'ONLY_IN_B'
        WHEN a.amount <> b.amount AND a.status <> b.status
            THEN 'AMOUNT_AND_STATUS_MISMATCH'
        WHEN a.amount <> b.amount THEN 'AMOUNT_MISMATCH'
        WHEN a.status <> b.status THEN 'STATUS_MISMATCH'
        ELSE 'MATCH'
    END
FROM a
FULL OUTER JOIN b ON a.record_id = b.record_id;

IF EXISTS
(
```

```
            SELECT record_id
            FROM gold.reconciliation
            GROUP BY record_id
            HAVING COUNT(*) > 1
        )
            THROW 50004, 'Published record keys must be unique.', 1;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

The transaction groups the Gold replacement and its uniqueness check. If an error occurs, the replacement must not leave a committed empty or partial Gold result. The error is rethrown so a caller can detect failure rather than interpreting it as success.

This is a deliberately small teaching pattern. It rejects empty inputs because the lab expects two populated snapshots; a production contract must distinguish a legitimate empty extract from a failed delivery. It publishes duplicates and invalid amounts as visible exceptions but blocks blank keys. These policies must be approved by the business owner for a real workload.

The supplied `@BatchId` labels the publication; it does **not** prove that both sources belong to that batch. Before executing, a supervisor must confirm that the intended source pair is fully loaded and that no refresh is running. Keep source loading and reconciliation serialized in the lab. Production requires durable source/batch control and an appropriate concurrency policy.

6. Execute, inspect, and repeat

After confirming the completed source pair, execute:

```sql
EXEC gold.usp_refresh_reconciliation
    @BatchId = 'training-2026-10-28-baseline';

SELECT record_id, a_amount, b_amount, classification
FROM gold.reconciliation
ORDER BY record_id;

SELECT classification, COUNT(*) AS record_count
FROM gold.reconciliation
GROUP BY classification
ORDER BY classification;
```

The expected baseline is:

Record ID	Classification	Explanation
A001	MATCH	Identifier formatting and status case normalize; amounts agree
A002	AMOUNT_MISMATCH	System A has 200.00; System B has 250.00
A003	ONLY_IN_A	No System B record
A004	ONLY_IN_B	No System A record
A005	DUPLICATE_KEY	System A contains two rows; neither is selected as a winner
A006	INVALID_AMOUNT	System A cannot supply a usable decimal amount

There are six result rows, one match, and five exceptions. The eleven input rows do not imply eleven business records. The published grain is one row per distinct normalized nonblank key across both sources.

Execute the same procedure again without changing the sources. The same six business classifications should remain. The publication timestamp can change; the result must not grow to twelve rows.

This illustrates **idempotency**: repeating work for the same intended input state yields the same intended business state. It does not mean every operational field must remain byte-for-byte identical.

Practice: follow a correction through the layers

Predict the result if System B A002 changes from 250.00 to 200.00. Then, if time and access permit, make that change in the synthetic source file, reload the complete source pair, and rerun the procedure.

The expected result is still six rows, now with two matches and four exceptions. A002 becomes `MATCH`; the other classifications remain unchanged.

Restore the baseline file and reload it before later lessons if they will use the baseline totals. Do not patch the Gold result directly: that would bypass the transformation and erase the connection between source evidence and published output.

Lesson conclusion

We have moved from source-shaped text to consistent values, explicit quality rules, and a repeatable business result. The most important lesson is that data quality and reconciliation are part of the transformation contract, not cleanup activities to hide before a report is shown.

Previous: Lesson 2
Next: Lesson 4: Consumption architecture

Product references: T-SQL surface area and Warehouse transactions. The examples require target-tenant preflight; this document does not claim they have been executed in GCC High.

LESSON 04 / 35 MINUTES

Consumption architecture

> **WHAT YOU WILL PRODUCE**
>
> A selected consumption path with a documented rationale.

What this lesson teaches

Producing a correct table does not automatically produce a governed analytical experience. Consumers still need agreed definitions, appropriate access, understandable measures, and confidence that they are looking at the intended publication.

This lesson connects the curated reconciliation output to Power BI. We will distinguish the role of a semantic model from the storage/query mode it uses, compare model-based reporting with direct SQL consumption, and decide which responsibilities belong upstream versus in the model.

Use approximately eight minutes for the conceptual distinctions, fifteen for the worked model/report, seven for governance and freshness, and five for the consumption decision.

1. Separate business semantics from data access

The Gold reconciliation table has already answered a business question at a defined grain: one row per normalized record ID for the current comparison. It exposes the amounts, statuses, source row counts, and reconciliation classification.

A **semantic model** adds the reusable analytical contract around that data. It defines measures, relationships where needed, meaningful names, formatting, and model-level access rules. Reports connect to the model to ask analytical questions without each report author inventing a new interpretation of the underlying columns.

For example, the organization needs a consistent definition of an exception. In this lesson:

> An exception is any published reconciliation record whose classification is not `MATCH`.

That definition includes duplicate keys and invalid amounts. If one report excludes them while another includes them, the two reports can show different exception rates even though both read the same Gold table.

The semantic model is where reusable reporting measures can express that agreed definition. The underlying record-level classification remains in Gold because it is also needed by SQL consumers and other downstream tools.

This division avoids two opposite mistakes. We should not reimplement the reconciliation algorithm independently in every report, but we also should not move every display-oriented measure into a physical SQL table.

2. Understand the storage/query choices

A shared semantic model and Direct Lake are not competing architectures. A shared model can use Direct Lake tables. It can also use Import or DirectQuery, subject to the model design and available capabilities.

The choice of storage/query mode describes how table data is made available to the model's query engine. The choice to share a model describes how business definitions are reused.

Mode or route	How to understand it	Important operating consideration
Import	Data is loaded into the semantic model's imported storage	Source changes require a successful model refresh before the imported data changes
DirectQuery	Model queries are translated into queries against the source	Source performance, connection behavior, and query-time permissions matter
Direct Lake	Model tables use supported Delta data in OneLake, loading needed data into memory	Model framing/freshness, supported source objects, capacity, and identity behavior still matter
Direct SQL	A SQL client queries the Warehouse through SQL access	SQL permissions and curated definitions must be governed independently of report access

Direct Lake avoids the conventional full import of the analytical dataset into model storage. It does not mean that no data enters memory, that every source object is supported, or that freshness no longer needs attention.

Current general documentation distinguishes **Direct Lake on OneLake** from **Direct Lake on SQL**. Their source discovery, security interactions, and fallback behavior differ. Direct Lake on SQL can fall back to DirectQuery in supported circumstances, including SQL views, depending on its behavior settings. Direct Lake on OneLake does not use that DirectQuery fallback path.

For this lesson, use the physical `gold.reconciliation` table and inspect the actual mode in the prepared model. Do not label a model "Direct Lake" merely because its data originated in Fabric. Verify the available experience in the target Government tenant rather than assuming both variants are present.

Import remains a legitimate option if it is the verified path and meets the workload. Choosing a working, supportable mode is more important than displaying a particular product label.

3. Build a small, reusable reconciliation model

Connect to the Gold result using the preflighted tenant experience and create or inspect an explicitly named semantic model. Do not depend on the assumption that creating a Warehouse automatically creates the intended model and measures.

For the examples below, name the model table `Reconciliation`. Map it to `gold.reconciliation`, and expose friendly names where useful. Retain `batch_id` and `published_at_utc` so consumers can understand which publication they are viewing.

The current table is sufficient for a small lesson. A broader enterprise model might separate facts and dimensions, but adding those structures here would distract from the consumption decision.

Define reusable measures

Create these measures in the semantic model:

DAX

```
Compared Records =
COUNTROWS('Reconciliation')
```

DAX

```
Matched Records =
COALESCE(
    CALCULATE(
        [Compared Records],
        KEEPFILTERS('Reconciliation'[classification] = "MATCH")
    ),
    0
)
```

DAX

```
Exception Records =
COALESCE(
    CALCULATE(
        [Compared Records],
        KEEPFILTERS('Reconciliation'[classification] <> "MATCH")
    ),
    0
)
```

```dax
Match Rate =
DIVIDE([Matched Records], [Compared Records])
```

`Compared Records` counts Gold business keys, not raw source rows. `KEEPFILTERS` makes the classification condition intersect the current report filters rather than replacing a classification selection. This matters when a visual or slicer already limits the classifications being shown.

Format `Match Rate` as a percentage. In an unfiltered baseline report, expect six compared records, one match, five exceptions, and a match rate of approximately 16.67%.

The current Gold table contains only the latest publication. A chart using its timestamp does not create a historical trend. Historical analysis would require a deliberate history design rather than pretending overwritten snapshots remain available.

Create an explainable report

Use summary cards for the four measures, a classification breakdown, and a detail table containing record ID, both amounts, both statuses, classification, and publication metadata.

The summary tells the audience where to look. The detail table lets them investigate why.

Trace A002 through the report and back to SQL. Its mismatch should be explained by 200.00 versus 250.00, not by a visual effect or a report-only formula. Trace A005 as well: a duplicate is an exception even though its source amounts appear consistent.

Do not make a match rate look better by filtering away quality exceptions without disclosure. If an approved business metric uses only comparable records as its denominator, define that as a separate measure with a distinct name and documented eligibility rules.

4. Compare direct SQL consumption

Some users need SQL rather than a report. An analyst may investigate exceptions, or an approved integration may read the curated output.

Direct SQL is reasonable when those consumers have the necessary skills, permissions, and support agreement. They should read the curated result rather than rebuilding reconciliation independently from Bronze.

For example:

```sql
SELECT
    batch_id,
    classification,
    COUNT(*) AS record_count
FROM gold.reconciliation
GROUP BY batch_id, classification
ORDER BY batch_id, classification;
```

This produces a classification summary without duplicating the record-level comparison rules.

However, SQL clients do not automatically inherit the DAX measures or semantic-model row-level security. Granting SQL access is a separate decision. If direct SQL consumers must share an aggregate definition, expose and govern that definition through an approved SQL object or contract as well.

Similarly, a report author who needs to build a new report over a shared model may require model Build permission. That is not the same as permission merely to view an existing report, and neither should be assumed to require broad workspace write access.

5. Govern access and prove freshness

Access crosses several boundaries: source connections, workspace roles, Warehouse objects, the semantic model, and the report or app. There may also be OneLake access routes. Permission at one boundary is not proof that all other boundaries are appropriately restricted.

Test the intended consumer identity, not just an administrator's view. A successful demonstration under the presenter's broad privileges proves little about a read-only user's experience.

Separate four freshness questions:

1. Did both intended source snapshots load successfully?
2. Did the SQL publication complete for the intended comparison?
3. Did the chosen model mode observe that publication through its required refresh, framing, or query path?
4. Does the report display the intended publication and expected results?

For Import, the model refresh is an explicit additional data-copy step. For Direct Lake, inspect the applicable framing and freshness behavior instead of assuming every report interaction sees the latest source state immediately. For DirectQuery, query-time access still does not prove that the underlying Gold publication is the correct one.

Use the batch identifier, publication timestamp, and known expected result together. A timestamp by itself does not prove business correctness.

6. Make the consumption decision

Complete a short decision:

> Our primary audience is ___. We will publish ___ through a shared semantic model using the verified ___ mode. Record-level reconciliation rules remain in ___. Reusable reporting measures live in ___. Direct SQL access is permitted for ___ through ___. Freshness is verified by ___. The model and access owners are ___.

A strong decision identifies who owns definitions, who receives each access path, and how new publications become visible. "We will use Direct Lake because it is faster" is insufficient without testing the relevant workload, capability, and security requirements.

Lesson conclusion

Governed consumption is more than connecting a report to a table. We have separated the reusable business model from the storage/query mode, kept reconciliation rules consistent across consumers, and made freshness and access testable.

Previous: Lesson 3
Next: Lesson 5: Operations and GCC High findings

Product reference: Direct Lake overview. General documentation describes product options, not verified availability or identity behavior in the target GCC High environment.

LESSON 05 / 25 MINUTES

Operations and GCC High findings

> **WHAT YOU WILL PRODUCE**
>
> An initial operations checklist and findings register.

What this lesson teaches

A solution is operational when someone other than its original author can determine whether it ran correctly, understand what failed, and take the next approved action. Scheduling is part of that story, but ownership, monitoring, access, and lifecycle are equally important.

This lesson turns the reconciliation flow into an operating agreement. We will also distinguish a skill gap from a data problem or an environment finding. That distinction protects both the learner and the accuracy of the technical record.

Use seven minutes for scheduling and monitoring, five for access and ownership, five for naming and lifecycle, five for findings, and three to complete the operating checklist.

1. Schedule the process, not just an activity

The process is a chain of dependent states:

TEXT

```
Ready source pair
    -> completed ingestion
    -> validated source pair
    -> completed SQL publication
    -> current semantic model
    -> verified consumer result
```

Each arrow represents a condition that must be met, not merely a delay between clock times.

If ingestion runs at 6:00 AM and SQL runs at 6:15 AM, that schedule assumes ingestion always completes within fifteen minutes. It does not detect an authentication failure, an unusually large file, or the absence of one source. A controlled flow uses success dependencies and data checks wherever the approved tooling supports them.

The engagement previously used direct Dataflow scheduling while pipeline invocation had an authentication or permissions issue. That can be a useful interim ingestion approach. It does not establish that the whole reconciliation process is automated.

If downstream steps remain supervised, write that plainly:

> Ingestion is scheduled. The operator confirms both snapshots and then executes reconciliation and the required model freshness step. End-to-end unattended orchestration remains open.

This statement is more useful than an optimistic diagram because it exposes the actual operating dependency.

2. Monitor technical success and business correctness

A green activity status answers a narrow question: the service completed the configured activity without reporting a failure. It does not prove that the correct files were selected, that the comparison period matches, or that the resulting exceptions make sense.

For our baseline, ingestion should produce six System A rows and five System B rows. Gold should contain six keys, one match, and five exceptions. These are expected values for this synthetic example, not permanent thresholds for a changing business source.

A production monitor should distinguish expected volume variation from suspicious changes. A sudden zero-row extract may be legitimate, but it must not be treated as legitimate simply because the query succeeded.

Use three kinds of evidence:

Evidence	What it establishes	What it does not establish alone
Activity status and refresh history	Configured work completed or reported an error	Correct source scope or business results
Counts and quality checks	Volume, key uniqueness, and exception conditions	Correct permissions or intended comparison period
Batch/source evidence and consumer checks	Intended source pair reached the intended audience	Every other control is functioning

Record a run identifier, source snapshot identifiers or approved source evidence, start/end times, relevant object names, expected/actual counts, outcome, and evidence links. A run record should identify the last successful published comparison as well as the failed attempt.

Do not put passwords, tokens, connection secrets, or sensitive source contents in logs. Error messages and screenshots may need sanitization before they are shared.

Respond at the correct failure boundary

If System B fails to load, stop before reconciliation. Repair or rerun the approved ingestion path, then confirm both inputs belong to the same intended comparison.

If SQL fails before commit, retain the prior committed Gold result and investigate. The example procedure rolls back its Gold replacement on failure, but that protection does not repair partially refreshed Bronze tables.

If Gold succeeds but the model freshness step fails, the SQL publication and report may describe different states. Record the distinction. Do not rerun all ingestion automatically when the failure is only in model refresh.

If the report loads but totals are unexpected, investigate the business rule, model filters, source scope, and publication metadata before declaring a platform incident.

Retries are safe only when the stage's behavior is understood. Full replacement and an idempotent SQL publication help, but they do not remove the need to verify batch alignment and prevent conflicting concurrent runs.

3. Define workspace roles and artifact ownership

Workspace roles govern collaboration over workspace contents. Artifact-specific permissions govern particular items or access paths. They overlap, but they are not interchangeable.

At a high level, workspace Administrators manage the workspace and access, Members have broad collaboration capabilities, Contributors create and modify content, and Viewers consume content. Exact permissions depend on the item and current service behavior; use the approved tenant access design rather than assigning elevated roles to everyone in a lab.

A report consumer generally should not receive workspace write permissions merely to read a report. A report author using a shared model may need model Build permission without becoming an owner of the upstream Warehouse. A connection owner has responsibilities that differ from the author of the SQL procedure.

Power BI row-level security also requires care: users with elevated workspace editing roles are not an appropriate stand-in for testing a restricted consumer. Model-level rules do not automatically govern separate SQL or OneLake access routes.

Assign owners by responsibility:

Responsibility	Operating question
Source owner	Who confirms that the intended extracts are complete and comparable?
Connection owner	Who maintains the approved authentication and any required gateway dependencies?
Ingestion owner	Who supports file selection, mapping, and refresh behavior?
SQL/data owner	Who maintains comparison rules and approves quality/publication policy?
Model/report owner	Who maintains shared measures, freshness behavior, and consumer access?
Support coordinator	Who triages failures and escalates unresolved environment findings?

One person may hold multiple responsibilities. The important point is that the responsibilities are explicit and have backup coverage. "The person who built it" is not a durable support model.

4. Use naming and lifecycle to make the flow understandable

Names should communicate purpose without depending on tribal knowledge. `bronze.system_a`, `silver.normalized_records`, and `gold.reconciliation` make the teaching layers visible. `gold.usp_refresh_reconciliation` makes the procedure's intended action visible.

These are examples, not a new naming standard for the tenant. Use existing approved workspace and artifact conventions when implementing the real workload.

Names alone do not provide isolation. A table called `test_reconciliation` is not protected from production consumers simply because its name contains "test." Establish the appropriate environment, access, and publication boundaries.

Lifecycle means controlling how a change moves from authoring to use. A business-rule change should be reviewed, exercised against known examples, approved by its owner, and accompanied by an understood rollback or recovery path.

For example, suppose the business changes the exception policy so invalid amounts block publication. That is not a harmless visual adjustment. It changes procedure behavior, operating alerts, and the report's freshness during a failed run. The SQL, model documentation, and runbook must remain consistent.

Use version-controlled definitions and the approved promotion mechanism where available. Do not assume a particular Git integration or deployment feature works in Government merely because a commercial demonstration uses it.

Before deleting training artifacts, identify shared dependencies and obtain owner approval. A connection, Warehouse, or model created during a workshop may already be referenced elsewhere.

5. Separate learning needs from environment findings

Not every difficulty is a product issue, and not every product issue is a learning failure.

A **learning need** concerns a concept or skill: a participant cannot yet explain an outer join, distinguish procedure creation from execution, or identify why append duplicates a snapshot.

A **data issue** concerns the supplied data: a duplicate business key, an invalid amount, or a missing source extract. An expected quality exception is not necessarily a defect in the platform.

An **environment finding** concerns a reproducible capability, connection, permission, network, gateway, or service behavior in the specific environment.

Maintain a learning log separately from environment findings. Track data exceptions through the reconciliation/data-quality process rather than calling them Government limitations.

Learning-log example

> **Skill:** Interpreting duplicate keys before a join.
> **Evidence:** Participant expected A005 to be a match despite two System A rows.
> **Next practice:** Explain how a two-to-one join changes row count.
> **Owner:** Named participant and supporting technical lead.

Environment-finding template

Field	What to record
Observation	Narrow description of what failed
Scope	Tenant/workspace, relevant item type, connection, and tested execution role
Time and evidence	Timestamp, sanitized run/error identifier, approved evidence location
Reproduction	Minimal steps, expected behavior, and actual behavior
Impact	Which lesson or operating stage is blocked
Investigation status	Observed, under investigation, or confirmed with supporting evidence
Owner and next action	Named technical/support owner and next diagnostic or escalation
Interim approach	Approved alternative, manual effort, dependency gaps, and review condition

For the prior pipeline/Dataflow issue, do not write "GCC High does not support this" based solely on an authentication error. That error may reflect permissions, connection context, a service defect, or a capability boundary. Record the observed failure and investigation status until the cause is established.

General documentation, prior success in another tenant, and a successful run under a different identity are useful clues. None alone proves the path works for this workload's scheduled execution.

6. Complete the initial operations checklist

Write the actual answers rather than checking boxes without evidence.

Gate	Required operating answer
Source readiness	How the matching, complete extract pair is identified
Ingestion completion	Where refresh status and destination counts are checked
Comparison readiness	Who or what prevents mixed snapshots and overlapping runs
SQL publication	How the procedure is invoked, validated, and recovered on failure
Model freshness	Which mode-specific step or behavior is verified
Consumer verification	Which publication and expected business totals are visible
Access and ownership	Who supports each stage and approves changes
Findings	Where learning needs, data exceptions, and environment issues are routed

Practice: reason through an incomplete run

System A loads successfully, System B fails, and the previous day's report remains accessible. Explain the correct operating response.

The answer should stop the new comparison, retain and label the last successful publication, record the failed ingestion, and assign the next action. The accessible report is not evidence that today's run succeeded. After correction, verify a complete matching source pair before resuming.

Lesson conclusion

We can now describe not only how the solution works, but how it is operated when something goes wrong. The final lesson checks that the decisions, artifacts, evidence, and follow-ups are complete enough to hand to another practitioner.

Previous: Lesson 4
Next: Lesson 6: Exit review

Environment boundary: All automation and access paths require target-tenant validation. Do not present a documented interim approach as a verified production control.

LESSON 06 / 10 MINUTES

Exit review

> **WHAT YOU WILL PRODUCE**
>
> An evidence-based completion record, owned follow-ups, and participant track alignment.

What this lesson teaches

An exit review is not a recap of which Fabric screens were opened. It is a short acceptance conversation: what can participants explain, what was actually completed, what remains unresolved, and who will do the next piece of work?

This matters because a workshop can feel successful while leaving behind an implementation that nobody can repeat. Our goal is to make that gap visible without treating an environment blocker as a learner failure.

Use four minutes to confirm artifacts and conduct teach-back, four minutes to assign unresolved work, and two minutes to align follow-up learning tracks.

1. Confirm the artifacts, not just attendance

The blueprint defines five substantive outputs before this exit review. Inspect them together:

Lesson	Expected artifact	Evidence of a useful result
Architecture	Decision record	Explains the selected platform, rejected alternative, assumptions, and revisit condition
Ingestion	Refresh contract	Names the source scope, destinations, load mode, trigger, and downstream gate
SQL engineering	Curated objects and results	Explains classifications and demonstrates or clearly labels rerun behavior
Consumption	Consumption decision	Names the model strategy, actual mode, access path, and freshness check
Operations	Checklist and findings register	Provides ordered success gates, failure handling, and responsible owners

For each artifact, record one of four states:

Participant completed means the participant performed the work and can explain the result.

Presenter demonstrated means the live path was shown, but participant execution has not yet been established.

Prepared evidence reviewed means screenshots, scripts, or previously generated results were used instead of a live execution.

Blocked means the required outcome could not be established because a dependency remained unresolved.

These labels are not grades. They describe the evidence honestly. A prepared screenshot can preserve the lesson when a tenant issue occurs, but it cannot prove that the participant's connection works.

2. Use teach-back to reveal understanding

Ask participants to explain the flow in their own words. A strong explanation connects each component to its purpose:

> We selected Warehouse because this structured reconciliation workload needs relational transformations and is owned by a SQL-oriented team. Dataflow Gen2 selects the intended SharePoint files and replaces the source-aligned landing snapshots. SQL normalizes comparable values, identifies quality problems, and publishes one result per business key. A shared semantic model provides reusable reporting measures. The operating flow confirms source alignment and stage success before downstream publication.

Then probe the decisions with a few short questions.

Why is A005 an exception rather than a match? There are two System A rows for its key. The comparison contract does not authorize choosing one, even though their amounts agree.

Why does A006 not become zero? An unconvertible source value is missing valid numeric evidence, not a legitimate zero amount.

What happens when we rerun unchanged inputs? Replace-mode ingestion preserves the same six and five landing rows. The procedure replaces Gold with the same six classifications instead of appending duplicates.

Why is a direct dataflow schedule not necessarily an end-to-end solution? It starts ingestion but does not prove downstream SQL execution, model freshness, or success dependencies.

What would justify a notebook later? A demonstrated requirement for an appropriate pro-code capability, such as custom Python processing or Spark engineering, rather than the desire to include another tool.

Do not require participants to memorize syntax. We are checking whether they can explain the consequences of the design.

3. Review unresolved questions and assign owners

A question without an owner is unlikely to become an action. Convert each unresolved item into a specific next step with an accountable person, expected evidence, and an agreed review point.

For example, "Pipelines are broken" is too broad. A useful action is:

> Retest pipeline invocation of the named training dataflow using the approved connection and intended execution identity. Capture the sanitized run result, confirm the investigation status, and update the environment finding before unattended scheduling is proposed.

Likewise, "Need more SQL training" is not a sufficiently focused learning action. Replace it with:

> Practice executing the reconciliation procedure twice and explain why the output remains one row per key.

Use this completion record:

Item	Current state	Owner	Next action	Completion evidence	Review point
Architecture decision	Record actual state	Assign person	Resolve assumptions or approve decision	Updated decision record	Agree date/ event
Repeatable ingestion	Record actual state	Assign person	Validate outstanding source/destination path	Refresh result and counts	Agree date/ event
SQL publication	Record actual state	Assign person	Complete rerun or recovery walkthrough	Expected result and execution evidence	Agree date/ event
Consumptio n	Record actual state	Assign person	Verify mode, freshness, or consumer identity	Model/report evidence	Agree date/ event
Operations/ findings	Record actual state	Assign person	Close the highest-impact operating gap	Updated checklist or finding	Agree date/ event

Do not fill this table with invented commitments. The relevant people must accept ownership.

Keep source-data exceptions separate from unresolved implementation work. A005 being classified as a duplicate is the expected lesson result, not necessarily a SQL defect requiring a code fix.

4. Align the follow-up tracks

The blueprint separates deeper Warehouse/SQL engineering from notebook/Spark engineering. Track selection should reflect the work each participant will own, not a ranking of which tool is more advanced.

The **SQL/Warehouse track** fits practitioners responsible for relational modeling, stored procedures, repeatable full/incremental processing, quality rules, and SQL operations.

The **notebook/Spark track** fits selected engineers responsible for Python/PySpark transformation, Delta engineering, parameterized notebooks, and their orchestration.

Participants can have a primary track and later gain familiarity with the other. They do not all need Python to consume a governed semantic model, and notebook engineers still need to understand shared data contracts.

Record each participant's intended responsibility, preferred follow-up track, and one concrete skill to develop. Confirm this with the participant and technical lead rather than assigning tracks based only on today's tool familiarity.

5. Close with a repeatability standard

The strongest next demonstration is for another practitioner to repeat the scenario using the agreed guide and approved artifacts, without relying on the original presenter to remember hidden steps.

That does not mean every production gate has been satisfied today. This session establishes a repeatable learning implementation and makes the remaining operating and environment work explicit.

Close with the outcome:

> We have a reasoned architecture, a defined ingestion contract, explicit reconciliation rules, a consumption decision, and an initial operating model. Where we demonstrated rather than independently completed a path, that distinction is recorded. Where a dependency remains unresolved, it has an owner and an evidence-based next action.

Previous: Lesson 5